

Sistema de Gestión — Clínica Odontológica

Entrega 3: Excepciones, Colecciones Avanzadas y Persistencia CSV

Programación Orientada a Objetos — UADE 2026

Santiago Weinbinder - Mateo Galluzzo - Paolo Maffei

1. Descripción General

La tercera entrega extiende el sistema desarrollado en las entregas anteriores incorporando cuatro conceptos fundamentales del paradigma orientado a objetos: manejo de excepciones chequeadas mediante una jerarquía personalizada, uso de la Stream API de Java para operaciones funcionales sobre colecciones, implementación de la interfaz Comparable para definir un ordenamiento natural en la clase Paciente, y persistencia de datos entre ejecuciones mediante archivos CSV procesados con BufferedReader y BufferedWriter.

La arquitectura en capas establecida en la Entrega 2 — model, repository, service, ui y main — se mantiene íntegra y se extiende con dos nuevos paquetes: exception, que aloja la jerarquía de excepciones, e io, que contiene las clases de persistencia. La capa service fue refactorizada completamente para reemplazar el patrón de retorno booleano con propagación de excepciones chequeadas, haciendo explícitos los contratos de error de cada operación. El resultado es un sistema completamente operativo que preserva su estado entre ejecuciones y comunica errores de negocio de forma precisa y estructurada.

2. Jerarquía de Excepciones Personalizadas

ClinicaException

Clase base de la jerarquía de excepciones del sistema. Extiende directamente de Exception, lo que la convierte en una excepción chequeada: el compilador obliga a todo código que pueda lanzarla a declararla en su firma con throws o a atraparla explícitamente. Recibe un mensaje descriptivo como único parámetro y lo propaga mediante super. Su rol es unificar todas las excepciones de negocio bajo un tipo común, permitiendo que el ClinicaController capture cualquier error del sistema con un único bloque catch sin perder la posibilidad de distinguir subtipos cuando se requiere mayor granularidad.

PacienteNoEncontradoException

Se lanza cuando una operación de búsqueda, modificación o eliminación no encuentra un paciente con el identificador o DNI solicitado. Es producida por los métodos buscarPorId, buscarPorDni, eliminarPorId y actualizar de PacienteServiceImpl, así como por obtenerHistorialClinico cuando el paciente referenciado no existe en el repositorio.

OdontologoNoEncontradoException

Se lanza bajo las mismas circunstancias que su análogo de paciente, pero aplicada al dominio de odontólogos. Es producida por OdontologoServiceImpl cuando el ID o la matrícula solicitados no corresponden a ningún registro existente en el repositorio.

TurnoYaReservadoException

Se lanza cuando se intenta registrar o modificar un turno en una fecha y hora que el odontólogo ya tiene ocupada con otro turno activo. La verificación excluye explícitamente los turnos en estado CANCELADO, ya que estos liberan el horario. También excluye el propio turno al momento de actualizar, evitando que el sistema rechace un turno comparándolo consigo mismo.

DatoInvalidoException

Se lanza cuando un dato ingresado no supera las validaciones del sistema antes de ser persistido. Cubre dos dominios distintos: validaciones de paciente — nombre y apellido solo letras, DNI de siete u ocho dígitos, email con formato válido, fecha de ingreso no futura, domicilio completo — y validaciones de turno — fecha no anterior a hoy, hora dentro del rango de atención entre las ocho y las diecinueve horas con cincuenta y nueve minutos, y horarios alineados a intervalos de treinta minutos. También es lanzada por crearOdontologo cuando el tipo de especialista ingresado no corresponde a ninguna de las dos especialidades reconocidas por el sistema.

3. Stream API y Comparable<T>

Uso de Streams en los repositorios

Los tres repositorios incorporan métodos de búsqueda y filtrado implementados mediante la Stream API de Java. RepositorioPaciente expone buscarPorDni, que aplica filter seguido de findFirst yorElse para obtener el primer paciente cuyo DNI coincide, y listarOrdenadosPorApellido, que invoca sorted sobre el stream aprovechando el ordenamiento natural definido por Comparable. RepositorioOdontologo agrega buscarPorMatricula con la misma estructura. RepositorioTurno incorpora tres métodos adicionales: buscarPorRangoFechas, que filtra usando isBefore e isAfter sobre LocalDate; filtrarPorOdontologo y filtrarPorPaciente, que filtran por identificador del profesional o del paciente respectivamente.

La verificación de disponibilidad de horario en TurnoServiceImpl utiliza anyMatch sobre la colección completa de turnos, evaluando en una sola expresión cuatro condiciones simultáneas: que el turno evaluado no sea el mismo que se está modificando, que no esté cancelado, que corresponda al mismo odontólogo, y que coincida exactamente en fecha y hora. Esta construcción detiene el recorrido en cuanto encuentra el primer turno que satisface todas las condiciones, sin continuar iterando innecesariamente.

Comparable<Paciente>

La clase Paciente implementa la interfaz Comparable parametrizada con su propio tipo. El método compareTo establece como criterio primario el apellido comparado sin distinción de mayúsculas mediante compareToIgnoreCase, y como criterio de desempate el nombre bajo el mismo tratamiento. Este ordenamiento natural es el que utiliza el método sorted del stream en listarOrdenadosPorApellido sin necesidad de pasar un Comparator explícito, resultando en una lista ordenada alfabéticamente que el sistema puede presentar directamente al usuario.

4. Persistencia CSV

El paquete io contiene tres clases estáticas — PersistenciaPaciente, PersistenciaOdontologo y PersistenciaTurno — responsables de la serialización y deserialización de las entidades del dominio hacia y desde archivos de texto plano en formato CSV con punto y coma como separador. Cada clase expone dos métodos públicos: guardar, que recibe la lista completa de entidades y la escribe en el archivo correspondiente, y cargar, que lee el archivo y popula el repositorio recibido por parámetro.

Ambos métodos utilizan BufferedWriter y BufferedReader respectivamente, envolviéndolos sobre FileWriter y FileReader. El buffer reduce la cantidad de operaciones de escritura y lectura sobre el disco al acumular datos en memoria y transferirlos en bloques, mejorando el rendimiento. El cierre de los recursos se garantiza mediante un bloque finally que se ejecuta independientemente de si ocurrió una excepción durante la operación, evitando que los archivos queden bloqueados por el sistema operativo.

La serialización convierte cada objeto en una línea de texto cuyos campos están separados por punto y coma. La deserialización aplica split con el separador y un límite negativo para preservar los campos vacíos del final de línea, convirtiendo cada segmento al tipo correspondiente mediante `parseInt`, `parseLong`, `LocalDate.parse`, `LocalTime.parse` o `EstadoTurno.valueOf` según el campo. Las líneas que no pueden parsearse correctamente son descartadas con un aviso, permitiendo que el sistema inicie con los datos válidos aunque el archivo contenga registros corruptos.

`PersistenciaTurno.cargar` recibe los tres repositorios como parámetros porque un turno almacena referencias a objetos `Paciente` y `Odontologo` completos, no solo identificadores. Al cargar cada turno, el método resuelve estas referencias buscando en los repositorios ya cargados por ID. Adicionalmente, cada turno cargado se agrega al `HistorialClinico` del paciente correspondiente, reconstruyendo así la información derivada que no se persiste en un archivo separado. Este comportamiento replica exactamente lo que `TurnoServiceImpl` realiza en tiempo de ejecución al registrar un turno nuevo.

El orden de carga en `Main` es obligatorio: primero pacientes, luego odontólogos y finalmente turnos, ya que estos últimos dependen de que los anteriores estén disponibles en memoria para resolver sus referencias. Cada repositorio implementa el método `cargarEntidad`, diferenciado del método `guardar` estándar en que no sobrescribe el identificador del objeto sino que lo preserva tal como fue almacenado en el CSV, ajustando el contador interno para que el próximo identificador generado no colisione con los existentes.

5. Diagramas de Secuencia UML

Alta de un nuevo turno

El diagrama modela el flujo completo que se activa cuando el usuario selecciona la opción de registrar un turno. Participan cinco capas: el usuario, VistaClinica, ClinicaController, TurnoServiceImpl junto con los servicios de paciente y odontólogo, y RepositorioTurno. La Vista solicita los datos del turno y los entrega al Controller encapsulados en un objeto DatoTurno. El Controller delega la operación a TurnoServiceImpl.registrarTurno, que internamente ejecuta dos grupos de operaciones.

El primer grupo, denominado crearTurno, consulta a PacienteServiceImpl y OdontologoServiceImpl para recuperar los objetos completos a partir de los identificadores recibidos. Si alguno no existe, la excepción correspondiente se propaga hasta el Controller, que la captura y muestra el error al usuario sin continuar. El segundo grupo, guardar, valida los datos del turno — fecha, horario y alineación de minutos — y verifica la disponibilidad del odontólogo mediante Stream.anyMatch sobre la colección de turnos existentes. Si ambas validaciones se superan, el turno se persiste en el repositorio y se agrega al historial clínico del paciente.

Búsqueda de paciente por DNI

El diagrama modela la búsqueda de un paciente utilizando el número de documento como criterio. La Vista solicita el DNI al usuario y lo entrega al Controller, que delega en PacienteServiceImpl.buscarPorDni. El servicio invoca al repositorio, que recorre la colección con filter().findFirst().orElse(null). El diagrama bifurca en dos alternativas: si el repositorio retorna un objeto Paciente, el servicio lo propaga al Controller y la Vista muestra sus datos; si retorna null, el servicio lanza PacienteNoEncontradoException, el Controller la captura y la Vista muestra el mensaje de error. En ambos casos el flujo concluye con una pausa antes de retornar al menú.

6. Refactorización de la Capa Service

La refactorización más significativa de esta entrega afecta a los tres servicios y a la interfaz IService. En la Entrega 2, los métodos declaraban retorno boolean e informaban errores imprimiendo directamente. En la Entrega 3, la interfaz IService declara throws ClinicaException en los cuatro métodos que pueden fallar, y cada implementación lanza la subclase específica que corresponde al tipo de error detectado.

PacienteServiceImpl reemplaza su método validarDatos de retorno booleano por uno de retorno void que lanza DatoInvalidoException ante cualquier campo inválido. El método guardar llama a validarDatos directamente y propaga la excepción sin atraparla. OdontologoServiceImpl aplica el mismo patrón, incorporando además crearOdontologo como método factory privado que construye la subclase correcta de Odontologo según el tipo seleccionado o lanza DatoInvalidoException si el valor no es reconocido. TurnoServiceImpl es

el más complejo porque sus operaciones pueden lanzar tres tipos distintos de excepción dependiendo del paso que falle: la creación del turno, la validación de datos o la verificación de disponibilidad.

El ClinicaController captura ClinicaException, el tipo base de la jerarquía, en un único bloque catch por operación. Esto elimina la necesidad de conocer el subtipo concreto en la capa de presentación: el mensaje descriptivo ya viene redactado por la clase que lanzó la excepción, y el Controller solo lo delega a VistaClinica para mostrarlo al usuario. Los métodos de la Vista que leen tipos numéricos o fechas aplican try-catch individuales por campo para distinguir errores de formato de los errores de negocio: un parseInt que falla indica que el dato no es un número entero, lo cual es responsabilidad de la Vista manejar; un DNI numérico pero fuera de rango es responsabilidad del Service detectar.

7. Decisiones de Diseño

La elección de CSV como formato de persistencia responde a los requisitos de la consigna y a la simplicidad de implementación sin dependencias externas. El separador punto y coma fue preferido sobre la coma para evitar conflictos con valores que podrían contener comas en campos de texto libre como direcciones. El uso de split con límite negativo garantiza que los campos vacíos al final de la línea — como la obra social cuando no está informada — sean preservados en el array resultante y no silenciosamente descartados.

El historial clínico no se persiste en un archivo separado porque es información derivada: puede reconstruirse completamente a partir de los turnos que ya están en turnos.csv. Esta decisión elimina la necesidad de mantener sincronizados dos archivos que representan la misma información desde perspectivas distintas, reduciendo el riesgo de inconsistencias.

La jerarquía de excepciones fue diseñada como chequeada — extendiendo Exception y no RuntimeException — para que el compilador obligue a todo el código que interactúa con los servicios a reconocer explícitamente los posibles caminos de error. Esta decisión hace visible en las firmas de los métodos qué operaciones pueden fallar y por qué razón, convirtiendo los contratos de error en parte de la interfaz pública de cada clase en lugar de dejarlos implícitos.

La búsqueda por rango de fechas en RepositorioTurno utiliza la negación de isBefore e isAfter en lugar de un hipotético isAfterOrEqual porque LocalDate no expone ese método directamente. La expresión !fecha.isBefore(inicio) && !fecha.isAfter(fin) es semánticamente equivalente a fecha >= inicio && fecha <= fin y es el idioma estándar en Java para intervalos cerrados sobre tipos temporales.

La máquina de estados de EstadoTurno impone transiciones estrictas que reflejan el ciclo de vida real de un turno médico: solo puede confirmarse desde estado pendiente, solo puede completarse desde confirmado, y tanto pendiente como confirmado pueden cancelarse. Los estados cancelado y completado son finales e irreversibles. Esta restricción se implementa en un único método privado estadoPermitido en TurnoServiceImpl, centralizando toda la lógica de transición en un solo lugar y evitando que reglas de negocio queden dispersas entre los métodos confirmarTurno, cancelarTurno y completarTurno.